Module: Apex & .NET Basics

Challenge 1: Map .NET Concepts to the Lightning Platform

AccountUtils:

```
public class AccountUtils {
   public static List<Account> accountsByState(String state){
      List<Account> accList=[SELECT Id, Name FROM Account WHERE BillingState = :state];
      return accList;
   }
}
```

Challenge 2: Understand Execution Context

AccountTriggerHandler:

```
public class AccountTriggerHandler {

    public static void CreateAccounts (List<Account> accList)
    {
       for(Account acc : accList)
       {
          if(acc.ShippingState!=acc.BillingState)
          {
             acc.ShippingState = acc.BillingState;
          }
       }
    }
}
```

AccountTrigger on Account:

```
trigger AccountTrigger on Account (before insert)
{
   if (Trigger.isBefore && Trigger.isInsert) {
 AccountTriggerHandler.CreateAccounts(Trigger.new);
 }
}
```

AccountTriggerTest:

```
@isTest
```

```
public class AccountTriggerTest {

   @isTest static void TestCreate200Records(){

      List<Account> accts = new List<Account>();
      for(Integer i=0; i < 200; i++) {
         Account acct = new Account(Name='Test Account ' + i, BillingState = 'CA');
         accts.add(acct);
      }
      Test.startTest();
      insert accts;
      Test.stopTest();
      System.assertEquals(200, [SELECT Count() FROM Account WHERE ShippingState = 'CA' ]);
   }
}
```

Module: Apex Basics & Database

Challenge 1: Get Started with Apex

```
public class StringListTest {

public static List<String> generateStringList(Integer N){

      List<String> TestList = new List<String>();

      for(Integer i=0;i<N;i++){

         TestList.add('Test '+ i);

         system.debug(TestList[i]);

      }

      return TestList;

   }

}
```

_____

Challenge 3: Manipulate Records with DML

```
public class AccountHandler{
   public static Account insertNewAccount(String accountName){
      Account acctToBeInserted = new Account(Name=accountName, AccountNumber='12345678');

      try{
         insert acctToBeInserted;
```

```
      }catch(DMLException e){
         System.debug('Inside DMLException catch ,error is ' + e.getMessage());
         acctToBeInserted = null;
      }
           return acctToBeInserted;
     }
}
```

_____

Challenge 4: Write SOQL Queries

```
public class ContactSearch {

   public static List<Contact> searchForContacts(String lastName, String maillingpostalCode){

      List<Contact> contactList = [Select Id,Name from Contact where
                        LastName =: lastName and
                        MailingPostalCode =: maillingpostalCode];
      return contactList;
   }

}
```

_____

Challenge 5: Write SOSL Queries

- **SOQL and SOSL Reference**

```
public class ContactAndLeadSearch {

   public static list<list< sObject >> searchContactsAndLeads(String lastName){

      list<list< sObject >> ContactLeadList = [ Find : lastName IN ALL FIELDS
                                 RETURNING contact(name),
                                 lead (Name) ];
     return ContactLeadList;
  }
}
```

press ctrl +E to create the name ending with Sumith

```
public class ContactAndLeadSearch {
```

```
    public static List<List<SObject>> searchContactsAndLeads(String searchTerm) {
        return [
            FIND :searchTerm
            IN NAME FIELDS
            RETURNING
                Contact(Id, FirstName, LastName),
                Lead(Id, FirstName, LastName)
        ];
    }
}
```

Module: Apex Triggers

Challenge 1: Get Started with Apex Triggers

```
trigger AccountAddressTrigger on Account (before insert, before update) {
    for(Account a: Trigger.New){
        if(a.Match_Billing_Address__c == true && a.BillingPostalCode!= null){
            a.ShippingPostalCode=a.BillingPostalCode;
        }
    }
}
```

Challenge 2: Bulk Apex Triggers

```
trigger ClosedOpportunityTrigger on Opportunity (after insert, after update) {
    List<Task> taskList = new List<Task>();
    for(Opportunity opp : [SELECT Id, StageName FROM Opportunity WHERE StageName='Closed
Won' AND Id IN : Trigger.New]){
        taskList.add(new Task(Subject='Follow Up Test Task', WhatId = opp.Id));
    }

    if(taskList.size()>0){
        insert tasklist;
    }
}
```

Module: Apex Testing

Challenge 1: Get Started with Apex Unit Tests

1. VerisfyDate

```
public class VerifyDate {

 //method to handle potential checks against two dates
 public static Date CheckDates(Date date1, Date date2) {
  //if date2 is within the next 30 days of date1, use date2.  Otherwise use the end of the month
  if(DateWithin30Days(date1,date2)) {
   return date2;
  } else {
   return SetEndOfMonthDate(date1);
  }
 }

 //method to check if date2 is within the next 30 days of date1
 private static Boolean DateWithin30Days(Date date1, Date date2) {
  //check for date2 being in the past
       if( date2 < date1) { return false; }

       //check that date2 is within (>=) 30 days of date1
       Date date30Days = date1.addDays(30); //create a date 30 days away from date1
  if( date2 >= date30Days ) { return false; }
  else { return true; }
 }

 //method to return the end of the month of a given date
 private static Date SetEndOfMonthDate(Date date1) {
  Integer totalDays = Date.daysInMonth(date1.year(), date1.month());
  Date lastDay = Date.newInstance(date1.year(), date1.month(), totalDays);
  return lastDay;
 }
```

```
}
```

2.TestVerifyDate

****

```
@isTest
public class TestVerifyDate
{
   static testMethod void testMethod1()
   {
      Date d = VerifyDate.CheckDates(System.today(),System.today()+1);
      Date d1 = VerifyDate.CheckDates(System.today(),System.today()+60);
   }
}
```

Challenge 2: Test Apex Triggers

1.restrictcontactbyname

****

```
trigger RestrictContactByName on Contact (before insert, before update) {

   //check contacts prior to insert or update for invalid data
   For (Contact c : Trigger.New) {
      if(c.LastName == 'INVALIDNAME') {    //invalidname is invalid
         c.AddError('The Last Name "'+c.LastName+'" is not allowed for DML');
      }
   }
}
```

2.testrestrictcontactname

****

```
@isTest
private class TestRestrictContactByName {

   static testMethod void  metodoTest()
   {
```

```
        List<Contact> listContact= new List<Contact>();
        Contact c1 = new Contact(FirstName='Francesco', LastName='Riggio' , email='Test@test.com');
        Contact c2 = new Contact(FirstName='Francesco1', LastName =
'INVALIDNAME',email='Test@test.com');
        listContact.add(c1);
        listContact.add(c2);

        Test.startTest();
          try
          {
             insert listContact;
          }
          catch(Exception ee)
          {
          }

        Test.stopTest();
    }
}
```

Challenge 3: [Create Test Data for Apex Tests]

```
//@isTest
public class RandomContactFactory {
    public static List<Contact> generateRandomContacts(Integer numContactsToGenerate, String
FName) {
        List<Contact> contactList = new List<Contact>();

    for(Integer i=0;i<numContactsToGenerate;i++) {
          Contact c = new Contact(FirstName=FName + ' ' + i, LastName = 'Contact '+i);
          contactList.add(c);
          System.debug(c);
        }
        //insert contactList;
        System.debug(contactList.size());
        return contactList;
    }

}
```

Module: [Find and Fix Bugs with Apex Replay Debugger]

Module: [Asynchronous Apex]

Challenge2: [Use Future Methods]

1.1 AccountProcessor

****

```
public class AccountProcessor {
   @future
   public static void countContacts(List<Id> accountIds){
      List<Account> accounts = [Select Id, Name from Account Where Id IN : accountIds];
      List<Account> updatedAccounts = new List<Account>();
      for(Account account : accounts){
        account.Number_of_Contacts__c = [Select count() from Contact Where AccountId =:
account.Id];
         System.debug('No Of Contacts = ' + account.Number_of_Contacts__c);
         updatedAccounts.add(account);
      }
      update updatedAccounts;
   }

}
```

1.2 AccountProcessorTest

****

```
@isTest
public class AccountProcessorTest {
   @isTest
   public static void testNoOfContacts(){
      Account a = new Account();
      a.Name = 'Test Account';
      Insert a;

      Contact c = new Contact();
      c.FirstName = 'Bob';
      c.LastName =  'Willie';
      c.AccountId = a.Id;

      Contact c2 = new Contact();
      c2.FirstName = 'Tom';
      c2.LastName = 'Cruise';
      c2.AccountId = a.Id;

      List<Id> acctIds = new List<Id>();
      acctIds.add(a.Id);

      Test.startTest();
      AccountProcessor.countContacts(acctIds);
      Test.stopTest();
   }

}
```

Challenge 3: Use Batch Apex

3.1 LeadProcessor

****

```
public class LeadProcessor implements Database.Batchable<sObject> {

    public Database.QueryLocator start(Database.BatchableContext bc) {
      // collect the batches of records or objects to be passed to execute
       return Database.getQueryLocator([Select LeadSource From Lead ]);
   }
   public void execute(Database.BatchableContext bc, List<Lead> leads){
       // process each batch of records
         for (Lead Lead : leads) {
            lead.LeadSource = 'Dreamforce';
         }
      update leads;
   }
   public void finish(Database.BatchableContext bc){
    }

}
```

3.2 LeadProcessorTest

****

```
@isTest
public class LeadProcessorTest {

      @testSetup
   static void setup() {
      List<Lead> leads = new List<Lead>();
      for(Integer counter=0 ;counter <200;counter++){
         Lead lead = new Lead();
         lead.FirstName ='FirstName';
         lead.LastName ='LastName'+counter;
         lead.Company ='demo'+counter;
         leads.add(lead);
      }
      insert leads;
   }

   @isTest static void test() {
      Test.startTest();
      LeadProcessor leadProcessor = new LeadProcessor();
      Id batchId = Database.executeBatch(leadProcessor);
      Test.stopTest();
   }

}
```

Challenge 4:Control Processes with Queueable Apex

AddPrimaryContact

****

```
public class AddPrimaryContact implements Queueable
{
   private Contact c;
   private String state;
   public  AddPrimaryContact(Contact c, String state)
   {
      this.c = c;
      this.state = state;
   }
   public void execute(QueueableContext context)
   {
      List<Account> ListAccount = [SELECT ID, Name ,(Select id,FirstName,LastName from contacts )
FROM ACCOUNT WHERE BillingState = :state LIMIT 200];
      List<Contact> lstContact = new List<Contact>();
      for (Account acc:ListAccount)
      {
            Contact cont = c.clone(false,false,false,false);
            cont.AccountId =  acc.id;
            lstContact.add( cont );
      }

      if(lstContact.size() >0 )
      {
         insert lstContact;
      }

   }
}
```

AddPrimaryContactTest

****

```
@isTest
public class AddPrimaryContactTest
{
    @isTest static void TestList()
    {
       List<Account> Teste = new List <Account>();
       for(Integer i=0;i<50;i++)
       {
          Teste.add(new Account(BillingState = 'CA', name = 'Test'+i));
       }
       for(Integer j=0;j<50;j++)
       {
          Teste.add(new Account(BillingState = 'NY', name = 'Test'+j));
```

```
        }
        insert Teste;

        Contact co = new Contact();
        co.FirstName='demo';
        co.LastName ='demo';
        insert co;
        String state = 'CA';

         AddPrimaryContact apc = new AddPrimaryContact(co, state);
         Test.startTest();
           System.enqueueJob(apc);
         Test.stopTest();
     }
 }
```

Challenge 5: [Schedule Jobs Using the Apex Scheduler]

DailyLeadProcessor

```
*****************
public class DailyLeadProcessor implements Schedulable  {
   Public void execute(SchedulableContext SC){
     List<Lead> LeadObj=[SELECT Id from Lead where LeadSource=null limit 200];
      for(Lead l:LeadObj){
         l.LeadSource='Dreamforce';
         update l;
      }
   }
}
```

DailyLeadProcessorTest

```
********************
@isTest
private class DailyLeadProcessorTest {
   static testMethod void testDailyLeadProcessor() {
      String CRON_EXP = '0 0 1 * * ?';
      List<Lead> lList = new List<Lead>();
      for (Integer i = 0; i < 200; i++) {
         lList.add(new Lead(LastName='Dreamforce'+i, Company='Test1 Inc.', Status='Open - Not
Contacted'));
      }
      insert lList;

      Test.startTest();
      String jobId = System.schedule('DailyLeadProcessor', CRON_EXP, new DailyLeadProcessor());
   }
}
```

Module Agent Customization with Apex

Challenge 1: Get Ready to Create an Apex Action

Challenge 2: Create an Apex Agent Action